ShelfEye Pi Setup Guide

Every command needed to set up a ShelfEye shelf camera, from a blank SD card to a running unit.

What you need

Item	Notes
Raspberry Pi Zero W or Zero 2 W	Must have Wi-Fi. The plain Pi Zero has none and will not work.
Camera Module 3 + Pi Zero camera cable	The Zero needs the narrow 22-pin to 15-pin cable.
SD card (16 GB or more)	Holds the system, the software, photos and the local database.
A computer with Raspberry Pi Imager	Download it from raspberrypi.com/software.
2.4 GHz Wi-Fi with internet	The Pi Zero cannot use 5 GHz networks. Internet is needed for OpenAI.
OpenAI API key	Goes in the config file in Step 3.
The ShelfEye server running	Its address (e.g. http://192.168.1.50:8000) and device token.

Step 1 Flash the SD card (on your computer)

- Open Raspberry Pi Imager. **Device:** your Pi Zero model. **OS:** Raspberry Pi OS (other) > **Raspberry Pi OS Lite** (32-bit for a Zero W; 64-bit is fine on a Zero 2 W). **Storage:** your SD card.
- When asked to apply OS customisation, choose **Edit settings** and set: hostname **shelfeye-1**, a username and password (write them down), your Wi-Fi name and password, Wi-Fi country **GH**.
- On the **Services** tab, turn on **Enable SSH** with password authentication.
- Click **Write** and wait for it to finish and verify.

> **Windows may say "You need to format the disk" after flashing. Click Cancel every time.** That is the Linux partition, which Windows cannot read. Formatting it would erase the system.

Step 2 Copy ShelfEye onto the SD card (on your computer)

Remove and reinsert the SD card. A drive called **bootfs** appears in File Explorer. Copy the project's **shelfeye/device** folder onto it and rename the copy to **shelfeye**. It should look like this:

```
bootfs/
  shelfeye/
    install.sh
    shelfeye.service
    shelfeye.env.example
    catalog.example.json
    shelfeye_device/        (the Python program)
```

Step 3 Create the two config files (on your computer)

Inside **bootfs/shelfeye**, copy **shelfeye.env.example** to a new file called **shelfeye.env** and fill in your values:

```
SHELFEYE_NODE_ID=shelf-1
OPENAI_API_KEY=sk-your-real-key
OPENAI_MODEL=gpt-4.1-mini
SHELFEYE_SERVER_URL=http://192.168.1.50:8000
SHELFEYE_DEVICE_TOKEN=a-long-random-secret
SHELFEYE_DATA_DIR=/home/pi/shelfeye-data
SHELFEYE_CATALOG=/etc/shelfeye/catalog.json
```

- **SHELFEYE_NODE_ID**: a unique name per camera (shelf-1, shelf-2, ...).
- **SHELFEYE_SERVER_URL**: your server's fixed IP or public address.
- **SHELFEYE_DEVICE_TOKEN**: must match the token the server was started with.
- Leave **/home/pi/** as it is. The installer replaces it with your real username.

Then copy **catalog.example.json** to **catalog.json** and list the products on this shelf, spelled the way you want them shown:

```
[
  "Paracetamol 500mg",
  "Amoxicillin 250mg",
  "Vitamin C 1000mg"
]
```

> Your API key sits on the SD card's readable partition until Step 5. The installer moves it to a protected file and deletes this copy.

Step 4 First boot and connect

Connect the camera cable, insert the SD card, and power on the Pi. Wait 2 to 3 minutes for the first boot and Wi-Fi. Then, from your computer (PowerShell or Terminal):

```
ssh yourusername@shelfeye-1.local
```

Type **yes** if asked to trust the device, then enter your password. If **shelfeye-1.local** is not found, look up the Pi's IP address in your router's connected devices list and use **ssh yourusername@<that-ip>**.

Step 5 Install everything (on the Pi)

This one command installs the packages, copies ShelfEye into place, secures the config and starts ShelfEye on every boot. It takes several minutes on a Pi Zero.

```
sudo bash /boot/firmware/shelfeye/install.sh
```

It ends with **Done**. For reference, these are the only packages it installs:

```
sudo apt update
sudo apt install -y --no-install-recommends python3-picamera2 python3-pil python3-requests
```

Step 6 Check that everything works (on the Pi)

Check	Command	You should see
1. Packages installed	`python3 -c "import picamera2, PIL, requests; print('all packages OK')"`	all packages OK
2. Camera detected	`rpicam-hello --list-cameras`	A camera named imx708
3. Camera takes a photo	`rpicam-still -o test.jpg && ls -lh test.jpg`	test.jpg, a few MB
4. Server reachable	`curl http://192.168.1.50:8000/api/nodes`	[] or a list
5. ShelfEye running	`systemctl status shelfeye`	active (running)
6. Watch it live	`journalctl -u shelfeye -f`	Count changes; Ctrl+C to exit

Finally, open the dashboard on your computer and take one item off the shelf. Within about two minutes the count drops and the change appears under **Recent activity**.

Everyday commands

To do this	Run this on the Pi
See whether ShelfEye is running	`systemctl status shelfeye`
Watch the live log	`journalctl -u shelfeye -f`
See the last 50 log lines	`journalctl -u shelfeye -n 50`
Edit the config (API key, server, timing)	`sudo nano /etc/shelfeye/shelfeye.env`
Edit the product list	`sudo nano /etc/shelfeye/catalog.json`
Apply config changes	`sudo systemctl restart shelfeye`
Stop or start ShelfEye	`sudo systemctl stop shelfeye` `sudo systemctl start shelfeye`
See stored photos	`ls ~/shelfeye-data/images`
Check free space on the SD card	`df -h /`
Restart the Pi	`sudo reboot`
Shut down safely	`sudo shutdown now`

> **Always run sudo shutdown now and wait for the green light to stop blinking before flipping the rocker switch off.** Cutting power while the Pi is writing can corrupt the SD card.

If something goes wrong

Problem	Fix
ssh: could not resolve shelfeye-1.local	Wait another minute, or use the Pi's IP address from your router. Check the Wi-Fi name, password and 2.4 GHz band set in Imager.
Package install fails	The Pi has no internet. Check Wi-Fi, then run Step 5 again.
No cameras available	Power off, reseat the ribbon cable (contacts facing the board on the Pi Zero end), then power on.
curl to the server fails	Check the server is running with --host 0.0.0.0, the IP is right, and port 8000 is allowed in the server's firewall.
Service is not running	Run journalctl -u shelfeye -n 50. Usually a missing API key or a typo in shelfeye.env.
"401 Invalid device token" in the log	SHELFEYE_DEVICE_TOKEN on the Pi does not match the server's token.
OpenAI errors (401 or 429)	401: the API key is wrong. 429: the account has no credit or hit its rate limit.